Contents

Java interview guide, part 1: core language and runtime

This is a guide to the interview, not just answers. Each question opens with what the interviewer is really testing and closes with the follow-ups they will ask next. Coworker-coaching voice throughout. This is **Part 1 of 3**.

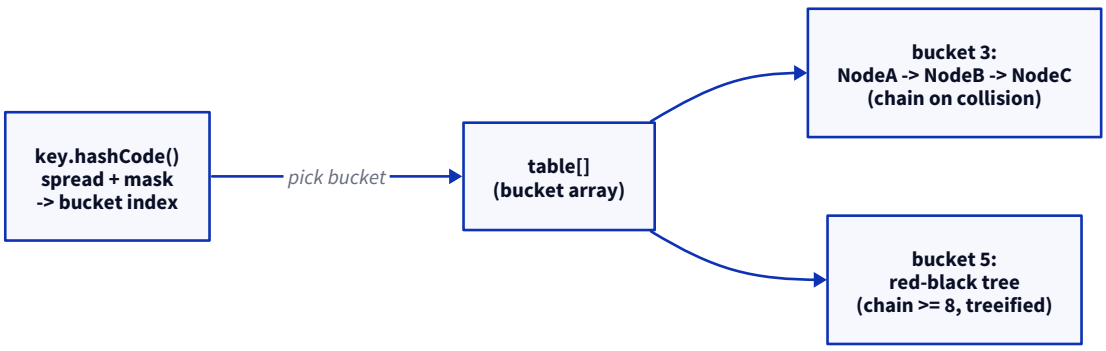
Core Java

1. Explain how a HashMap works internally.

What they are testing: whether you understand hashing, collisions, and Big-O in practice, not just "it stores key-value pairs." A weak answer ("it uses a hash") invites the follow-up "what happens on a collision" and "why is get O(1)."

Answer. A `HashMap` is an **array of buckets**, and the key's `hashCode()` decides which bucket a key-value pair goes in. On `put`, it takes `key.hashCode()`, spreads the bits (to mix high bits down), and masks it to a bucket index. If the bucket is empty, it drops the entry there; if the bucket already has entries (a **collision**), it appends to that bucket. Reads work the same way: hash to the bucket, then walk it comparing with `equals()`.

The depth that earns marks: within a bucket, entries start as a **linked list**, and since Java 8, once a bucket grows past **8 entries (and the table is at least 64)**, it **treeifies** into a red-black tree, so a pathological bucket degrades to O(log n) instead of O(n). The table **resizes** (doubles) when size exceeds `capacity * loadFactor` (default 0.75), rehashing entries.



So lookups are **O(1) on average**, O(log n) worst case for a treeified bucket. The whole thing hinges on a good `hashCode()`, which is why the next question is always equals/hashCode.

Follow-ups they will ask.

- *Why load factor 0.75?* A balance between space and collision rate, higher packs tighter but collides more.

- *What if hashCode() is terrible (all same value)?* Every entry lands in one bucket, so you get O(log n) after treeify (pre-Java-8 it was O(n)).
- *Why treeify at 8?* Statistically, with a decent hash, a bucket rarely reaches 8, so treeify only kicks in when the hash is bad.

2. Why is HashMap not thread-safe?

What they are testing: whether you can name the concrete failure, not just repeat "it's not thread-safe." The follow-up for a vague answer is "what actually goes wrong?"

Answer. Because its `put`, `resize`, and read paths have **no synchronisation**, so concurrent writes corrupt it. Two threads putting at once can **lose an entry** (both write the same bucket slot), and a concurrent **resize** can leave the internal structure inconsistent. Historically (pre-Java-8) a concurrent resize could form a **circular linked list** in a bucket, so a later `get` would **spin forever at 100% CPU**, a real production outage people have hit. Java 8 changed the resize so it does not cycle, but `HashMap` is still unsafe: you can lose updates and see corrupted state.

The misconception to correct: "I only read concurrently, so it is fine." Not true if any thread writes, because a resize triggered by a write can happen under a reader.

Follow-ups they will ask.

- *How do you make it safe?* `ConcurrentHashMap` (preferred), or `Collections.synchronizedMap` (locks every call).
- *Is `Collections.synchronizedMap` enough?* For single operations yes, but a compound action (check-then-put) still races, use `putIfAbsent` / `compute`.

3. Difference between HashMap and ConcurrentHashMap.

What they are testing: whether you know *how* `ConcurrentHashMap` achieves thread safety cheaply, not just that it is thread-safe.

Answer. Both are hash maps; the difference is concurrency. `HashMap` is **unsynchronised** (fast, single-threaded). `ConcurrentHashMap` is **thread-safe with fine-grained locking**: it does not lock the whole map, it locks (or CAS-updates) only the bucket being written, so many threads write different buckets in parallel. **Reads are lock-free**. Practical differences worth naming:

	HashMap	**ConcurrentHashMap**
Thread-safe	No	Yes
Null keys/values	Allows one null key, null values	**No nulls at all**
Locking	None	Per-bin (fine-grained)
Iterator	Fail-fast (throws on concurrent mod)	Weakly consistent (no throw)

The "no nulls" point is a classic gotcha, and the reason is that a null return from `get` would be ambiguous under concurrency (absent v/s present-but-null).

Follow-ups they will ask.

- *Why does it forbid null?* To disambiguate `get` returning null in a concurrent setting.
- *Is `size()` exact?* It is an estimate under concurrent writes, which is fine by design.

- *`synchronizedMap` v/s `ConcurrentHashMap`*? The former locks the whole map per call; the latter locks per bucket, so it scales far better under contention.

4. Explain the equals() and hashCode() contracts.

What they are testing: whether you understand *why* the contract exists, because breaking it silently breaks every hash-based collection. Weak answers get "what breaks if you override one but not the other?"

Answer. The contract ties the two together: **if two objects are equal by `equals()`, they must return the same `hashCode()`.** The reverse is not required (equal hashcodes do not imply equality, that is just a collision). `equals()` must also be reflexive, symmetric, transitive, and consistent.

Why it matters: hash collections use `hashCode()` to find the **bucket** and `equals()` to find the entry **within** it. So if you override `equals()` but not `hashCode()`, two "equal" objects can land in **different buckets**, and a `HashMap` lookup with a logically-equal key returns null, the object is "lost" in the map. This is one of the most common real bugs.

```
// Broken: equals overridden, hashCode not -> HashMap lookups fail
record Point(int x, int y) {}   // records generate BOTH correctly, prefer them
```

Follow-ups they will ask.

- *What if two unequal objects share a hashCode?* Fine, that is a collision, `equals()` sorts them out within the bucket.
- *Mutable fields in hashCode?* Dangerous, if a key's hashCode changes after insertion, you cannot find it again.
- *How do records help?* They auto-generate both consistently, removing the whole class of bug.

5. Why is String immutable?

What they are testing: whether you connect immutability to concrete benefits (security, caching, thread safety), not just state the fact.

Answer. A `String` cannot change after creation; any "modifying" method returns a **new** String. It was made immutable on purpose, and the reasons are the benefits: **the String Pool can safely share one instance** across many references (only safe because it never changes); **thread safety for free** (no locking to read a String); **safe as a `HashMap` key** (its hashCode cannot change under the map); **security** (a String passed to a file path or a connection cannot be altered after a security check); and **cached hashCode** (computed once, since it never changes).

Follow-ups they will ask.

- *How is it enforced?* The class is `final` (no subclass can add mutability) and the backing array is private.
- *Then how does `concat` work?* Returns a new String; the original is untouched.
- *Downside?* Building a String in a loop with `+` creates garbage each iteration, use `StringBuilder`.

6. Explain the String Pool.

What they are testing: memory model understanding and the `==` v/s `equals` trap.

Answer. The String Pool (String Constant Pool) is a **region of the heap where the JVM keeps one copy of each distinct String literal** and reuses it. So two identical **literals** point to the **same** object, which is why `"a"`

`== "a"` is true. A `new String("a")` deliberately creates a **separate** heap object, so `"a" == new String("a")` is false even though the values match. `intern()` forces a String into the pool.

The interview point this sets up: **always compare Strings with `equals()`, never `==`**, because `==` compares references and only accidentally works for pooled literals, then fails on runtime-built Strings.

Follow-ups they will ask.

- *Where does the pool live?* On the heap since Java 7 (was PermGen before), so interned Strings can be GC'd.
- *When would you call `intern()`?* Rarely, only for a bounded set of heavily-repeated values; over-interning bloats the pool.

7. What is Type Erasure in Generics?

What they are testing: whether you know generics are compile-time only, which explains a whole set of "why can't I" limitations.

Answer. Type erasure means **generics exist only at compile time**: the compiler checks types, then **removes them**, so at runtime `List<String>` is just a raw `List`. It replaces each type parameter with its bound (or `Object`) and inserts the casts for you. Java did this for **backward compatibility** with pre-generics code.

The consequences are the real content: you **cannot** do `new T()`, `new T[]`, or `instanceof List<String>` (no type info at runtime), and you **cannot use primitives** as type arguments (they erase toward `Object`, so you use `Integer`).

Follow-ups they will ask.

- *Why can't you overload `method(List<String>)` and `method(List<Integer>)`?* After erasure both are `method(List)`, same signature.
- *Does it cost performance?* No, it is compile-time only, no runtime overhead.
- *How do you get around no `new T()`?* Pass a `Class<T>` or a `Supplier<T>` in.

8. Difference between ArrayList and LinkedList, and when would you actually use each?

What they are testing: whether you reason from data structures to real performance, or just recite Big-O. The "actually use" is a trap, they want to hear that ArrayList wins in practice.

Answer. `ArrayList` is backed by a **resizable array**; `LinkedList` is a **doubly-linked list of nodes**.

Operation	ArrayList	LinkedList
Index access `get(i)`	**O(1)**	O(n)
Add/remove at end	Amortised O(1)	O(1)
Add/remove at head	O(n)	O(1)
Memory	Compact, cache-friendly	Node + 2 pointers per element

The honest real-world answer: **use `ArrayList` almost always.** Even though `LinkedList` has O(1) insert in the middle *if you already hold the node*, you usually have to **traverse to find the position** (O(n)), and its poor cache locality makes it slower in practice than the Big-O suggests. `LinkedList` only wins as a **queue/deque** with lots of head/tail operations, and even there `ArrayDeque` is usually better.

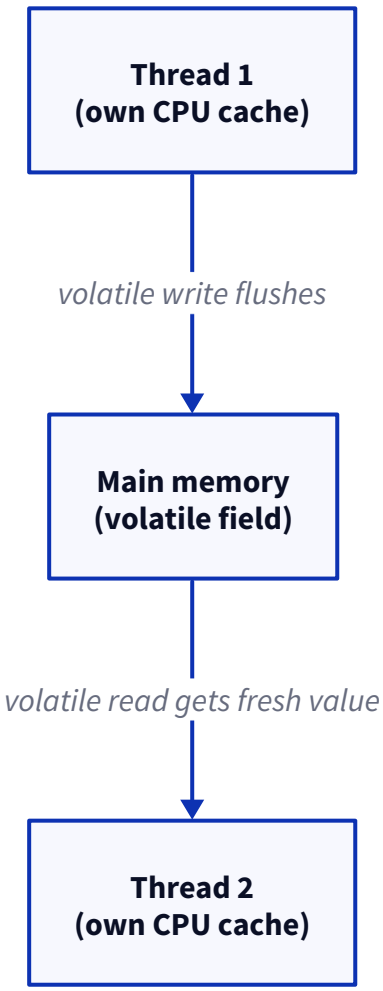
Follow-ups they will ask.

- *Why is ArrayList faster despite O(n) inserts?* Cache locality, contiguous memory is far friendlier to the CPU than chasing pointers.
- *When is LinkedList genuinely right?* Rarely, prefer `ArrayDeque` for queue/stack needs.

9. Explain volatile.

What they are testing: whether you understand the **visibility** problem in the memory model, not just "it makes a variable thread-safe" (which is wrong).

Answer. `volatile` guarantees **visibility and ordering** for a single variable: a write by one thread is **immediately visible** to others (it goes to main memory, not just the writing thread's CPU cache), and it prevents the compiler/CPU from reordering around it. Without it, one thread may **never see** another's update because of caching, the classic "flag that never flips" bug.



The critical limitation to state before they ask: **`volatile` does NOT make compound operations atomic.** `count++` on a volatile is still a race (read-modify-write is three steps). So `volatile` is for **visibility** (a status flag one thread sets and another reads), not for **mutual exclusion** (use `synchronized` or an atomic for that).

Follow-ups they will ask.

- *So can I use `volatile` for a counter?* No, `count++` still races; use `AtomicInteger`.
- *Good use case?* A `volatile boolean running` flag to stop a thread.
- *Does it lock?* No, it is cheaper than a lock, just no mutual exclusion.

10. Difference between volatile and synchronized.

What they are testing: whether you know they solve **different** problems (visibility v/s mutual-exclusion-plus-visibility).

Answer. `volatile` gives you **visibility and ordering for one variable**, no locking, no atomicity for compound actions. `synchronized` gives you **mutual exclusion** (one thread in the block at a time) **and** visibility (a happens-before guarantee on entry/exit). So:

	volatile	**synchronized**
Visibility	Yes	Yes
Mutual exclusion	**No**	Yes
Atomic compound ops	No	Yes
Cost	Cheap	More expensive (lock)
Scope	One variable	A block/method

Rule of thumb: use `volatile` for a **simple flag** read/written across threads; use `synchronized` (or a lock, or an atomic) when you have a **compound action** or need to guard a section.

Follow-ups they will ask.

- *Which is faster?* `volatile`, but it does less; do not use it where you need exclusion.
- *Alternative to synchronized for counters?* `AtomicInteger` (lock-free CAS).

11. Explain the Java Memory Model.

What they are testing: senior-level understanding of *why* concurrency needs rules at all. This separates people who memorised `synchronized` from people who understand it.

Answer. The Java Memory Model (JMM) defines **when a write by one thread is guaranteed to be visible to another**, and what reorderings are allowed. The core idea is **happens-before**: if action A happens-before action B, then A's effects are visible to B. Without a happens-before relationship, the CPU and compiler are free to **cache and reorder**, so one thread may see stale or out-of-order values.

The relationships that create happens-before: **unlocking a monitor** happens-before locking it (that is what `synchronized` gives you), a **`volatile` write** happens-before a subsequent read of it, `Thread.start()` happens-before the thread runs, and a thread's actions happen-before another thread's `join()` returns. So `synchronized`, `volatile`, and the concurrency utilities are not magic, they are the tools that **establish happens-before**.

Follow-ups they will ask.

- *Why does reordering exist?* Performance, the CPU and compiler optimise aggressively; the JMM says which optimisations are visible across threads.
- *What is a data race?* Two threads access the same variable, at least one writes, with no happens-before between them, that is undefined and where visibility bugs live.

12. Difference between Process, Thread, Concurrency, and Parallelism.

What they are testing: clean fundamentals and whether you know concurrency and parallelism are not the same thing (a favourite gotcha).

Answer. Quick and precise:

Term	Meaning
Process	An independent program with its **own isolated memory**; heavy
Thread	A unit of execution **inside** a process, **sharing** its memory; light
Concurrency	**Dealing with** many tasks at once by interleaving (progress overlaps, even on one core)
Parallelism	**Doing** many tasks at the same instant on **multiple cores**

The line that lands: **concurrency is about structure (many things in progress), parallelism is about execution (many things literally running at once).** You can have concurrency on a single core (interleaving); you need multiple cores for true parallelism. A concurrent program may or may not run in parallel.

Follow-ups they will ask.

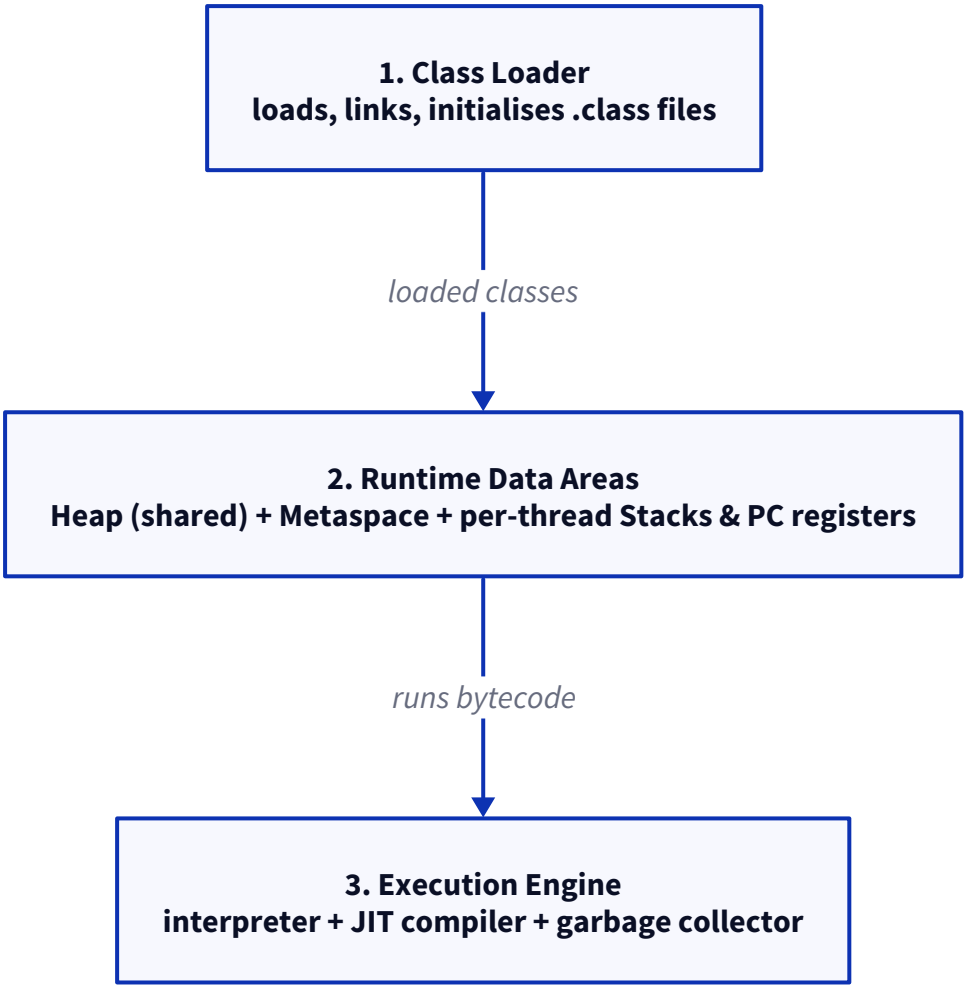
- *Can you have concurrency without parallelism?* Yes, one core time-slicing between tasks.
- *Threads share memory, processes do not, what does that imply?* Threads communicate cheaply but need synchronisation; processes are isolated but communicate via IPC.

JVM and memory

13. Explain JVM architecture.

What they are testing: whether you have a mental model of what runs your code, which underpins every GC and memory question. A senior is expected to name the three layers.

Answer. Three parts. The **Class Loader** loads, links, and initialises `.class` files into memory. The **Runtime Data Areas** hold the state: the **Heap** (all objects, shared across threads), **Metaspace** (class metadata), and, **per thread**, a **JVM Stack** (method frames, locals) and a **PC register**. The **Execution Engine** runs the bytecode, starting with the **interpreter**, promoting hot code with the **JIT compiler**, and reclaiming memory with the **garbage collector**.



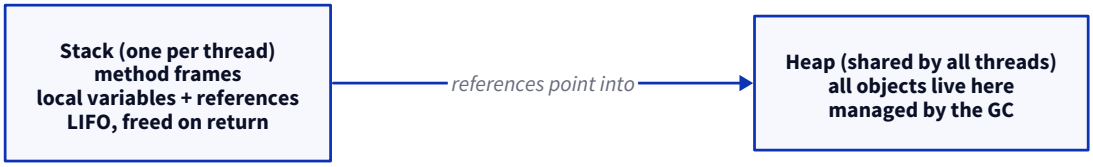
The point that shows understanding: the **heap is shared** (so it needs GC and is where thread-safety matters), while **each thread has its own stack** (so locals are inherently thread-safe).

Follow-ups. *What is the JIT?* It compiles frequently-run bytecode to native code at runtime for speed. *Interpreter v/s JIT?* Interpreter starts fast, JIT is faster once code is hot; the JVM uses both.

14. Heap v/s Stack.

What they are testing: where objects v/s locals live, and why it matters for threading and memory errors.

Answer. The **stack** is per-thread and holds **method frames**: local variables, method parameters, and references. It is LIFO and freed automatically when a method returns. The **heap** is **shared across all threads** and holds **all objects**; it is managed by the GC. A local variable sits on the stack, but the object it points to sits on the heap.



This directly explains the two memory errors: `StackOverflowError` (too-deep recursion fills a thread's stack) v/s `OutOfMemoryError` (the heap is full).

Follow-ups. *Why are locals thread-safe but object fields not?* Each thread has its own stack, but they share the heap. *Where do primitives live?* A local primitive is on the stack; a primitive field lives inside its object on the heap.

15. Metaspace v/s PermGen.

What they are testing: whether you know a specific modern JVM change; a quick knowledge check.

Answer. Both store **class metadata**. **PermGen** was the old fixed-size region (up to Java 7) that lived in the heap and caused the infamous `OutOfMemoryError: PermGen space` when you loaded too many classes. **Metaspace** (Java 8+) replaced it and lives in **native memory**, growing dynamically by default, so those PermGen OOMs largely went away. Short answer: Metaspace is the Java 8+ replacement for PermGen, moved off-heap.

16. Explain garbage collection.

What they are testing: conceptual grasp of automatic memory management and generational collection, not GC-algorithm trivia.

Answer. GC **automatically reclaims heap memory** for objects that are no longer reachable from any live reference (GC roots: stack references, statics, and so on). Modern JVMs use **generational** collection built on the observation that **most objects die young**: new objects go in the **Young generation** (collected often and cheaply by a minor GC), and survivors are promoted to the **Old generation** (collected less often by a slower major GC). So you get frequent cheap collections of short-lived garbage and rare expensive ones for long-lived objects.

Follow-ups. *What makes an object collectable?* Nothing reachable references it. *Does GC prevent memory leaks?* No, if you keep a reference (a growing cache, a static list), the object stays reachable and is never collected. *What is a stop-the-world pause?* GC pausing all app threads to collect; minimising it is the goal of modern collectors.

17. Which GC does Java 21 use by default?

What they are testing: current knowledge; a quick check.

Answer. G1 (Garbage-First), the default since Java 9. G1 splits the heap into regions and aims for **predictable pause times**. Worth naming the alternatives a senior might reach for: **ZGC** and **Shenandoah** for very low-

latency, large-heap workloads (sub-millisecond pauses), and **Parallel GC** when you want raw throughput over pause time.

18. How would you investigate an OutOfMemoryError?

What they are testing: whether you have actually debugged one, not read about it. Answer from method.

Answer. The way I work it: first read the **error message**, because it tells you the flavour, heap space v/s Metaspace v/s "GC overhead limit exceeded" v/s "unable to create native thread", and each points somewhere different. For a heap OOM I get a **heap dump** (run with `-XX:+HeapDumpOnOutOfMemoryError` so one is captured automatically) and open it in Eclipse MAT or VisualVM to find **what is holding the memory**, the dominator tree shows the biggest retainers, which is almost always a **growing collection or cache that never releases**. I also check **GC logs** to see whether the heap is genuinely full or just churning. So: read the message, capture a heap dump, find the top retainer, trace why it is still referenced.

Follow-ups. *OOM but heap looks fine?* Could be Metaspace (class-loader leak) or native threads. *Leak v/s just-too-small a heap?* A leak grows without bound; a small heap plateaus, bump `-Xmx` and see if it stabilises.

19. How would you investigate a memory leak?

What they are testing: the distinction between a leak and normal usage, and a methodical approach.

Answer. A Java "leak" is not unfreed memory (the GC handles that), it is **objects you unintentionally keep reachable**, so they are never collected. I confirm it is a leak by watching the heap over time: after each **full GC**, does used memory **keep climbing** instead of returning to a baseline? If yes, it is a leak. Then a **heap dump** and the dominator tree show what is growing, and I trace the **reference chain** back to the root holding it. The usual culprits: an **ever-growing static collection or cache** with no eviction, **unremoved listeners**, or **`ThreadLocal`s not cleaned up in a thread pool**.

Follow-ups. *Classic leak sources?* Static collections, caches without eviction, ThreadLocal in pools, unclosed resources. *Tool?* Heap dump + Eclipse MAT, or JFR over time.

20. How would you investigate high CPU usage?

What they are testing: a systematic method, and the thread-dump technique.

Answer. The reliable recipe: find the **hot thread**, then see what it is doing. On Linux I use `top -H` to find the **thread** (not just the process) burning CPU, note its **thread id**, convert it to hex, then take a **thread dump** (`jstack`) and find that thread, its stack trace shows the exact code running. If it is a tight loop or a spinning `HashMap` bug, it is right there. For a broader picture I use a **profiler or Java Flight Recorder**. So: hot thread first, then its stack, then the code. Common causes are a busy loop, excessive GC (check GC time), or a lock-free spin.

Follow-ups. *High CPU but low app work?* Likely GC thrashing, check GC logs. *Why hex thread id?* That is how the thread appears (`nid`) in the jstack output.

21. Which JVM tools have you used?

What they are testing: hands-on experience; a quick credibility check.

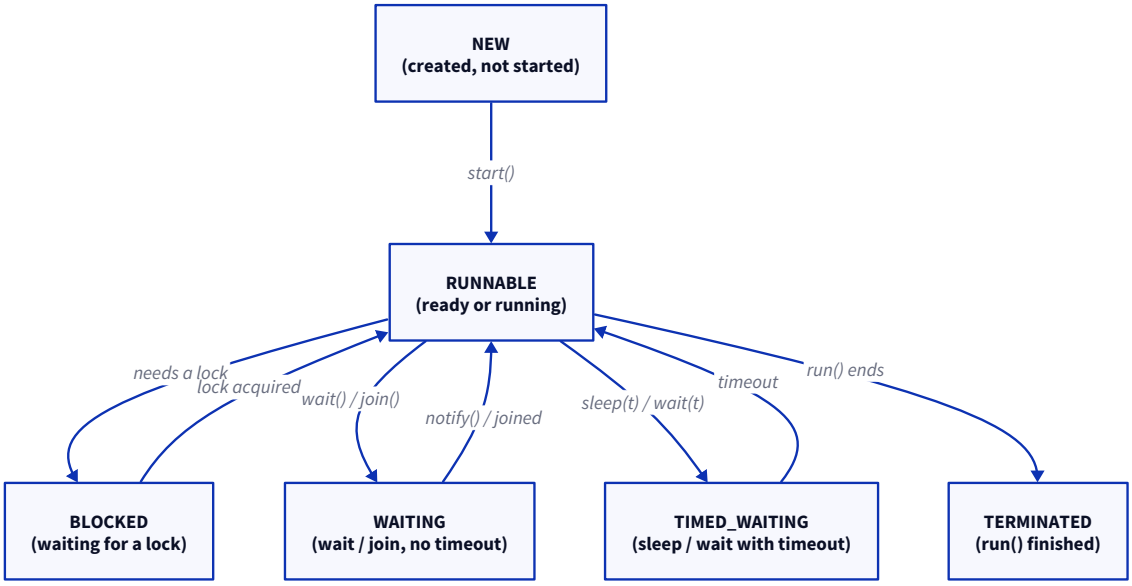
Answer. The ones I reach for: **`jstack`** for thread dumps (deadlocks, hot threads), **`jmap`** for heap dumps and histograms, **`jcmd`** as the modern all-in-one (dumps, GC info, JFR), **Java Flight Recorder** for low-overhead production profiling of allocations, CPU, and locks, and **VisualVM** or Eclipse MAT to explore heap dumps. `jstack` and a heap dump cover most incidents. *(Adapt to what you have actually used.)*

Multithreading

22. Explain the thread lifecycle.

What they are testing: whether you know the actual `Thread.State` values and transitions, not a vague "it runs then stops."

Answer. Six states: **NEW** (created, not started), **RUNNABLE** (eligible to run, covering both ready and actually running), **BLOCKED** (waiting for a monitor lock), **WAITING** (`wait()` / `join()` with no timeout), **TIMED_WAITING** (`sleep(t)`, `wait(t)`), and **TERMINATED** (finished). `start()` moves it from NEW to RUNNABLE; from there the OS scheduler and the locks/waits it hits drive the rest.



The subtlety worth mentioning: Java has **no separate RUNNING state**, RUNNABLE includes running, and the OS decides moment to moment who is on a CPU.

23. What is a race condition?

What they are testing: whether you can describe the mechanism precisely.

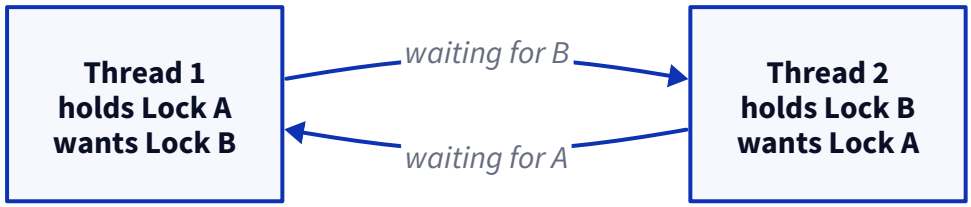
Answer. A race condition is when the **result depends on the timing of threads** accessing shared state, so the outcome is nondeterministic and sometimes wrong. The textbook case is `count++` on a shared variable: it is really **read, modify, write**, three steps, so two threads can both read the same old value and one update is lost. It happens whenever you have **shared mutable state** and no synchronisation. Fix it with a lock, `synchronized`, or an atomic.

Follow-ups. *Why is `count++` not atomic?* It compiles to read-modify-write. *Fix?* `AtomicInteger.incrementAndGet()` (lock-free CAS) or a synchronised block.

24. What is a deadlock?

What they are testing: the definition and, more importantly, how to prevent it.

Answer. A deadlock is when **two or more threads each wait for a lock the other holds**, so none can proceed, forever. The classic: Thread 1 holds Lock A and wants Lock B; Thread 2 holds Lock B and wants Lock A.



It needs four conditions together (mutual exclusion, hold-and-wait, no preemption, circular wait). The one you break in practice is **circular wait**: **always acquire locks in a consistent global order**, and no cycle can form. Other defenses: lock timeouts (`tryLock`) and holding fewer locks.

Follow-ups. *How do you prevent it?* Consistent lock ordering, mainly. *Detect at runtime?* A thread dump shows two threads each BLOCKED on a lock the other owns.

25. How would you debug a deadlock?

What they are testing: the practical technique.

Answer. Take a **thread dump** (`jstack` or `jcmd`). The JVM is helpful here: it **explicitly detects and prints "Found one Java-level deadlock,"** naming the two threads and which lock each holds and waits for. So you rarely have to hunt, the dump hands you the cycle, and you fix it by reordering the lock acquisition. If it is intermittent, take a few dumps while it is hung.

26. Explain thread starvation.

What they are testing: a quick concept check; keep it short.

Answer. Starvation is when a thread **never gets enough CPU or a lock** to make progress, because others keep taking it, often greedy high-priority threads, or a thread that keeps reacquiring a lock before a waiting one gets a turn. Unlike deadlock (stuck forever by mutual waiting), a starved thread *could* run, it just never gets the chance. Fair locks (`ReentrantLock(true)`) help.

27. Explain livelock.

What they are testing: whether you can distinguish it from deadlock; short.

Answer. In a livelock, threads are **not blocked but still make no progress**, they keep responding to each other and retrying, like two people stepping side to side in a corridor. The tell: a deadlocked thread is asleep (BLOCKED), a livelocked thread is **busy burning CPU** but achieving nothing. It often comes from naive "detect conflict, back off, retry" logic where everyone backs off in sync; random jitter breaks it.

28. Explain wait(), notify(), and notifyAll().

What they are testing: the coordination mechanism and the rules around it.

Answer. These live on `Object` and let threads coordinate. **`wait()`** makes a thread **release the lock and pause** until notified; **`notify()`** wakes **one** waiting thread; **`notifyAll()`** wakes **all** of them. You must call them **while holding the object's monitor** (inside `synchronized`), or you get `IllegalMonitorStateException`. Prefer **`notifyAll()`** when threads wait on different conditions, because `notify()` wakes an arbitrary one and might wake the wrong thread. In real code I would use a `BlockingQueue` or a `Condition`, but this is the primitive underneath.

29. Why should wait() always be inside a loop?

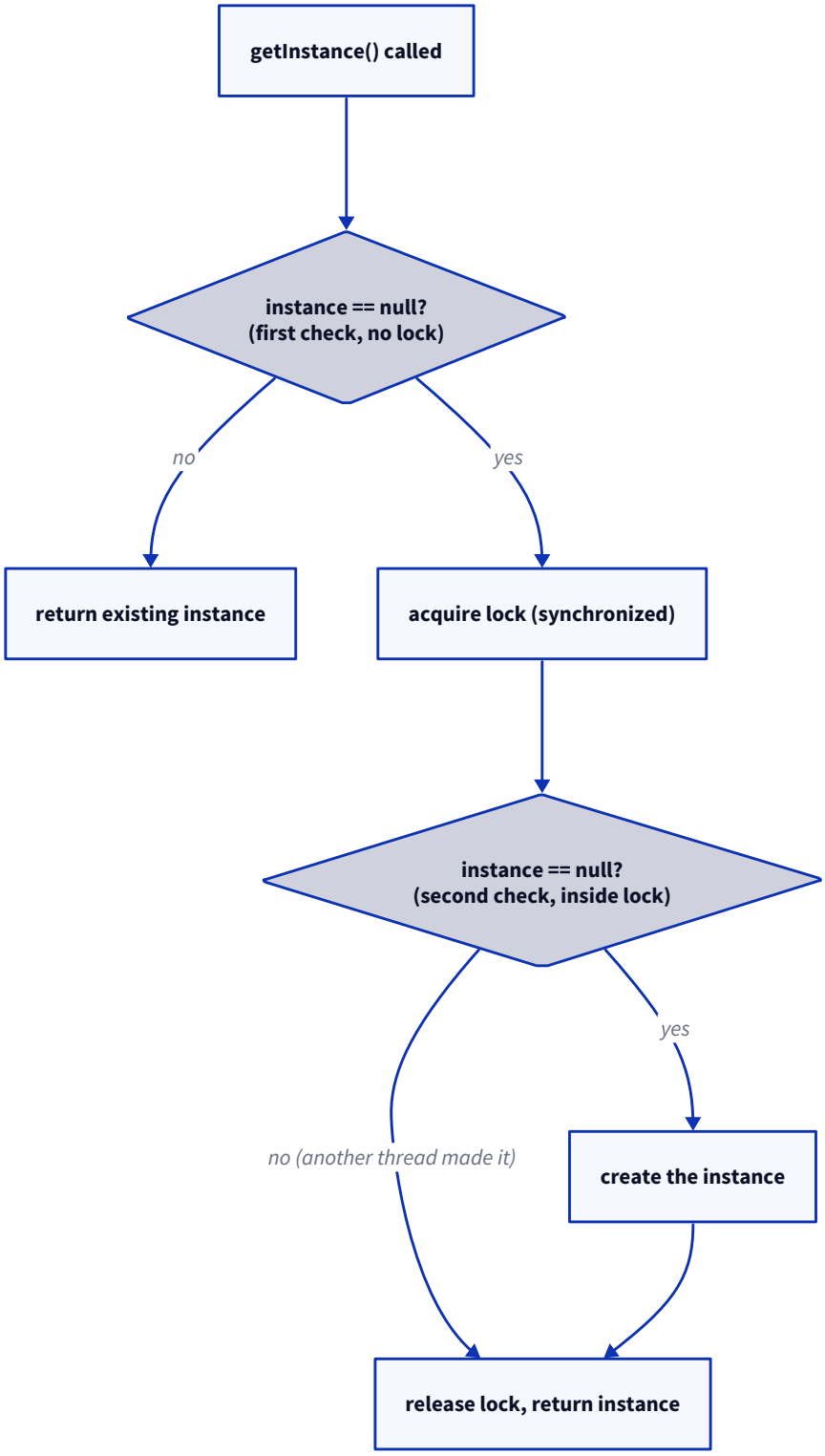
What they are testing: whether you know about spurious wakeups, a senior-level detail.

Answer. Two reasons, and the sharp one is **spurious wakeups**: the JVM is allowed to wake a waiting thread **without any notification**, so "I woke up" does not mean "the condition is true." The other is that even after a real notification, **another thread may have changed the condition** before this one reacquired the lock. So you re-check in a **`while` loop** (not `if`): `while (!condition) wait();`.

30. Explain double-checked locking, and why is volatile required?

What they are testing: a notorious concurrency puzzle that trips people who half-remember it.

Answer. Double-checked locking is a lazy-init optimisation: **check the field without a lock (fast path), and only if it is null take the lock and check again** before creating. So the lock is used only on the first creation, and every later call skips it.



The `volatile` **is mandatory**, and here is why: `instance = new Singleton()` is not atomic, it is allocate, assign the reference, run the constructor, and the JVM may **reorder** those. Without `volatile`, another thread could see a **non-null but half-constructed** object through the first (unlocked) check. `volatile` forbids that reordering and guarantees visibility. (Before Java 5 the memory model was too weak for even `volatile` to fix this, so DCL was simply broken then.) Honestly, I would use the **Bill Pugh holder idiom** instead, lazy, thread-safe, no volatile, no double check.

31. How do you stop a thread gracefully?

What they are testing: whether you know `Thread.stop()` is deprecated and the cooperative-interruption pattern, and this one warrants code.

Answer. You do **not** force-kill it (`Thread.stop()` is deprecated, it can leave state corrupt). The correct way is **cooperative**: signal the thread and let it stop itself, either with a `volatile` flag or by **interrupting** it and having the thread check the interrupt status.

```
class Worker implements Runnable {
    public void run() {
        // check the interrupt flag each loop
        while (!Thread.currentThread().isInterrupted()) {
            try {
                doWork();
            } catch (InterruptedException e) {
                // a blocking call was interrupted:
                // restore the flag and exit cleanly
                Thread.currentThread().interrupt();
                break;
            }
        }
        // cleanup here
    }
}
// elsewhere: worker.interrupt();  // asks it to stop
```

The key habit: **do not swallow** `InterruptedException` , either exit on it or restore the flag, or you lose the stop signal.

Follow-ups. *Why is* `Thread.stop()` *deprecated?* It kills the thread mid-operation, releasing locks with state half-updated. *volatile flag v/s interrupt?* Interrupt also wakes blocking calls (`sleep` , `wait`); a plain flag does not.